FINPILOT AI

FINANCE • CONTROL • DECIDE • GROW

AI-POWERED FINANCIAL DECISION & CONTROL PLATFORM

Comprehensive Technical Architecture, Implementation Guide, Interview Preparation & Viva Master Handbook (Parts 1 – 75)

PROJECT METADATA

Name: FinPilot AI
Core Stack: FastAPI • LangGraph • Razorpay • Tailwind

ARCHITECTURE LAYER

Design: Hybrid Deterministic + Agentic
Digital Twin: 90-Day Forward Engine

EVALUATION & SECURITY

RBAC: 4 Isolated Enterprise Roles
Audit: SHA-256 Chained Hash Ledger

TABLE OF CONTENTS

PART 1 — PROJECT OVERVIEW & PHILOSOPHY

What is FinPilot AI?

FinPilot AI is an enterprise-grade **Financial Digital Twin, Merchant Growth & Autonomous Multi-Agent Decision Operating System**. It acts as an autonomous AI Financial Controller that continuously monitors enterprise financial health, executes what-if forward simulations, audits invoices and payroll, reconciles accounts, and facilitates autonomous AI-to-AI commerce with Razorpay payment integration.

The Problem It Solves & Why It Is Important

Traditional Enterprise Resource Planning (ERP) systems and static BI dashboards are purely *retrospective*: they present data after money has already been spent, leading to unbudgeted overruns, duplicate vendor billings, and compliance penalties. Finance managers spend hundreds of hours manually cross-checking invoices against department budget utilization and verifying 18% GST tax math. FinPilot AI transforms finance from **reactive accounting to proactive simulation**.

Who Would Use It?

Chief Financial Officers (CFOs): Enterprise liquidity oversight, capital allocation what-if forecasting, and multi-million rupee disbursement authorization.

Finance Managers: Day-to-day budget tracking, vendor invoice approvals, payroll audits, and department burn-rate management.

Department Heads: Scoped budget monitoring, planned expense affordability simulation, and team allowance claims.

Auditors: Compliance verification, Form 16 vs. payroll reconciliation, duplicate transaction flagging, and immutable audit logs.

The Core Architectural Philosophy

```
DATA ──► RULES ──► ANALYSIS ──► RISK / EXCEPTION ──► DECISION ──► APPROVAL
```

Simple Explanation: Raw financial records (invoices, budgets) are verified by strict rules, processed by analytical algorithms, evaluated for risk, turned into actionable decisions, routed for human sign-off if large, executed on payment rails, and permanently logged to an immutable audit record.

Technical Explanation: Ingestion pipelines parse telemetry into Pydantic models; deterministic services execute constraint validations; the 7-agent orchestrator generates counterfactual trajectories against an in-memory Digital Twin; risk scoring triggers HITL state branches in LangGraph ($\ge ₹50,000$); authorized actions dispatch Razorpay payment links; and transactions are cryptographically chained via SHA-256 blocks.

PART 2 — READY-TO-SPEAK PROJECT EXPLANATIONS

⚡ 30-Second Elevator Pitch

"Most financial AI tools describe what already happened by reading spreadsheets. FinPilot AI simulates what happens next — before a human has to commit capital. Built on Razorpay test rails, it combines a 90-day Financial Digital Twin, a 7-agent autonomous copilot, merchant growth tools with smart upsells, and an AI-to-AI shopping protocol with strict role-based data isolation."

⏱️ 1-Minute Executive Summary

"FinPilot AI is an AI-powered financial decision platform designed for enterprise CFOs and finance teams. Traditional dashboards only show historical data after overspending occurs. FinPilot AI maintains a live in-memory Digital Twin that simulates day-by-day cash flows and runway under counterfactual what-if scenarios — like spending spikes or deferred payouts. It features an automated invoice auditor with 18% GST calculation and duplicate detection, an autonomous AI-to-AI commerce engine with Razorpay payment links, and a 7-agent reasoning pipeline that delivers 5-step role-tailored action plans with Human-in-the-Loop governance."

🎙️ 2-Minute Comprehensive Pitch

"I built FinPilot AI to solve the massive disconnect between static financial reporting and active financial decision-making. The system is architected around a hybrid model: deterministic Python logic handles exact arithmetic — like GST, budget balances, and statutory deductions — while autonomous AI agents handle intent classification, causal anomaly correlation, and what-if narration.

The platform features four key innovations: First, a 90-day Financial Digital Twin that evaluates liquidity and runway before transactions are approved. Second, a 7-agent LangGraph copilot that performs causal root-cause analysis on budget overruns. Third, a merchant growth studio with machine-readable catalogs (`/v1/commerce/ai-manifest`), conversational in-app checkout, and autonomous AI-to-AI negotiation bounded by spending tokens. Fourth, a zero-leakage role-based notification center with 2-way conversation threads and cryptographically chained SHA-256 audit logs."

🔬 5-Minute Deep Technical Walkthrough

"Let's dive into the technical architecture of FinPilot AI. The backend is built with Python 3.11, FastAPI, and Pydantic, while the frontend is a responsive Single Page Application with Tailwind CSS and Chart.js.

At the core is our Financial Digital Twin (`digital_twin_service.py`), which models liquid cash, department burn rates, and vendor liabilities. When a what-if query is executed, it runs differential day-by-day simulations applying custom multipliers without touching the database.

For reasoning, we implement a 7-agent orchestrator in `multi_agent_orchestrator.py` with LangGraph state machines: IntentAgent checks RBAC permissions; RetrievalAgent pulls role-scoped facts; AnalysisAgent calculates burn velocities; RiskAgent generates a 4-factor score; SimulationAgent runs forward models; CausalAgent correlates spending spikes with operational signals; and NarratorAgent formulates a 5-step executive report.

In commerce, our Universal Commerce Manifest exposes products in JSON-LD format. External AI buyers communicate with `/v1/commerce/ai-buy` with hard `X-Spending-Token-Limit` bounds. Transactions over ₹50,000 pause in LangGraph for Human-in-the-Loop approval, after which Razorpay test-mode payment links are dispatched and actions logged to our SHA-256 chained audit trail."

PART 3 — PROBLEM STATEMENT & SOLUTIONS

Operational Domain	Traditional Manual / Dashboard Failure	FinPilot AI Implementation Solution
Budget Monitoring	Departments realize they are over budget only at end-of-month reconciliation.	Real-time burn rate velocity tracking and forward day-by-day deficit prediction.
Invoice Auditing	Manual verification of tax calculations, vendor rates, and duplicate submissions.	Automated 18% GST validator, duplicate hash matching, and runway affordability check.
Reconciliation	Discrepancies between bank statements, general ledger, and payment gateway logs.	Automated 3-way reconciliation (Ledger vs. Bank vs. Razorpay) with exception logs.
Approval Workflows	Slow email chains without visibility into post-approval liquidity impact.	HITL Approvals Queue ($\ge ₹50K$ threshold) with 1-click Razorpay payment link dispatch.
Employee Allowances	Static caps create friction, leading to manual review fatigue and override backlogs.	Self-calibrating policy engine that analyzes override frequencies to propose adjustments.
Payroll & Form 16	Year-end TDS deduction mismatches and taxable salary discrepancies go unnoticed.	Automated Form 16 vs. payroll gross reconciliation with flagged review recommendations.
Merchant Sales	Slow checkout interfaces and inability to transact with autonomous AI agents.	Machine-readable store manifest (`/v1/commerce/ai-manifest`) and conversational in-app cart.

PART 4 — COMPLETE FEATURE MATRIX

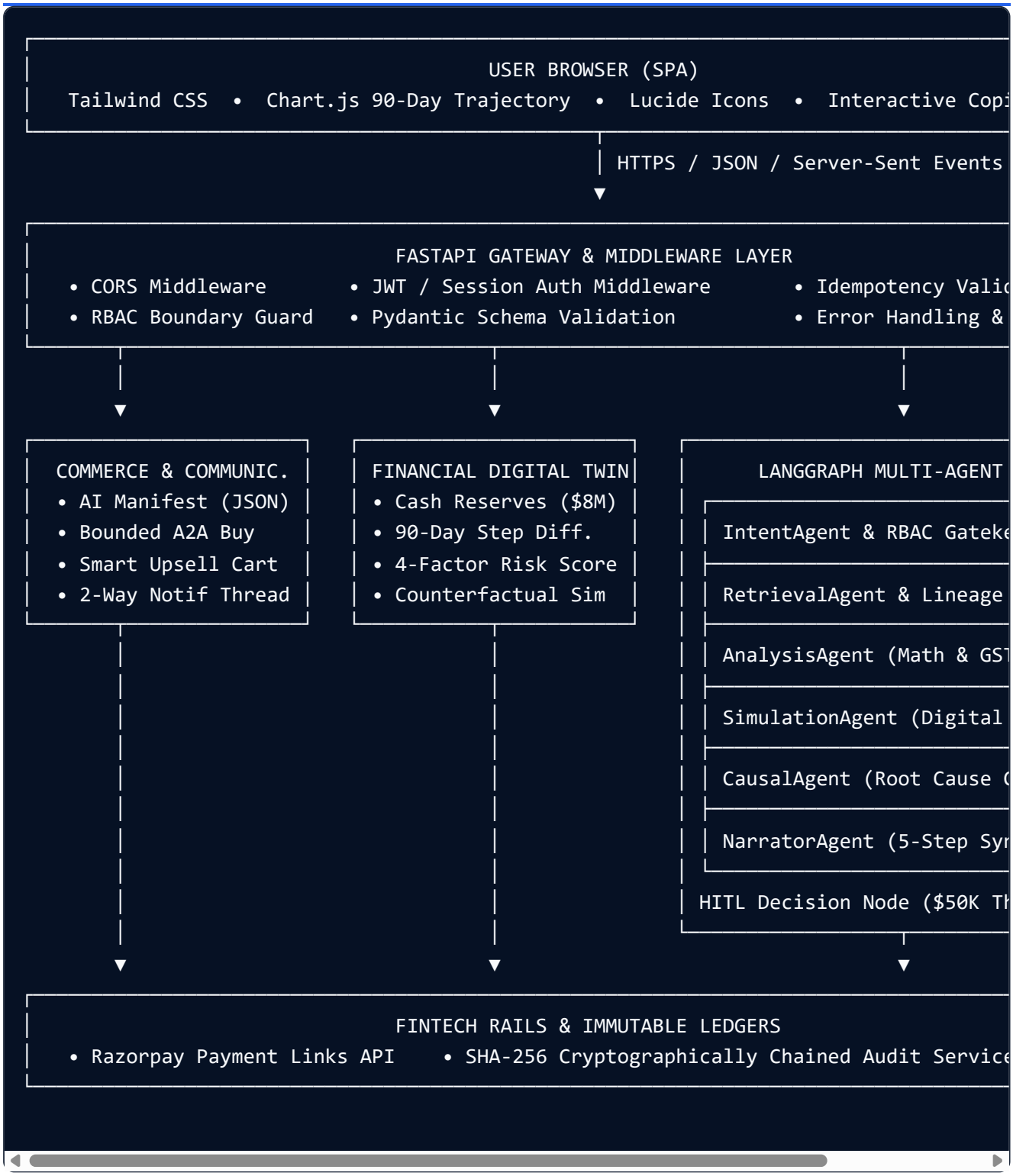
Feature	Purpose & Logic	Backend / Service	AI Involvement	Role Access	Status
Executive Control Center	Real-time liquidity, burn metrics, and decision scoring.	`intelligence_service.py`	Deterministic Telemetry	All (Role-scoped)	IMPLEMENTED
Financial Digital Twin Studio	90-day forward what-if trajectory simulation.	`digital_twin_service.py`	Differential Simulation	CFO, FM	IMPLEMENTED
Company Decision Map	Hierarchical financial tree (Root $ ightarrow$ Budget $ ightarrow$ Action).	`budget_service.py`	Graph Structure	CFO, FM, Auditor	IMPLEMENTED
Invoice Intelligence Auditor	GST 18% calculation, duplicate detection, affordability.	`invoice_service.py`	Fuzzy Match + Rules	All (Dept scoped)	IMPLEMENTED
HITL Approvals Queue	Disbursement governance ($\ge ₹50,000$ threshold).	`approvals.py`, `razorpay_service.py`	LangGraph HITL Node	CFO, FM	IMPLEMENTED
Financial Decision Copilot	Natural language reasoning & counterfactual analysis.	`multi_agent_orchestrator.py`	7-Agent Multi-Agent	All (RBAC Gate)	IMPLEMENTED
AI Commerce & Shopping Studio	Agent-readable store, A2A shopping, conversational cart.	`merchant_commerce_service.py`	Autonomous Buyer/Seller	All Roles	IMPLEMENTED
Role-Based Notification Center	User-targeted 2-way conversation threads & routing.	`notification_service.py`	Deterministic Routing	All Roles (Isolated)	IMPLEMENTED
Form 16 Tax Reconciler	Cross-checks payroll gross vs Form 16 TDS certificates.	`payroll_service.py`	Deterministic Math	CFO, FM, Auditor	IMPLEMENTED

Feature	Purpose & Logic	Backend / Service	AI Involvement	Role Access	Status
Employee Finance & Allowances	Salary revisions, allowance tracking, friction calibration.	`employee_finance_service.py`	Calibration Engine	CFO, FM, DeptHead	IMPLEMENTED
Immutable Audit Trail	SHA-256 cryptographically chained compliance logging.	`audit_service.py`	Cryptographic Hash	CFO, Auditor	IMPLEMENTED
Live Banking API Integration	Direct integration with real bank account webhooks.	`cash_flow_service.py`	None	Planned	PLANNED

PART 5 — TECHNOLOGY STACK & ARCHITECTURAL RATIONALE

Layer / Component	Selected Technology	Why It Was Chosen	Alternative Considered	Trade-Off Justification
Backend API Framework	**FastAPI (Python 3.11)**	Asynchronous performance, automatic OpenAPI `/docs` generation, strict Pydantic validation.	Django / Flask	FastAPI is substantially faster than Flask/Django for high-concurrency async endpoints and native typing.
Agentic Workflow State Machine	**LangGraph & LangChain Core**	Stateful graph coordination, cyclic agent hand-offs, checkpointing, and native Human-in-the-Loop interrupts.	Raw OpenAI API / AutoGen	LangGraph provides deterministic state transitions (`FinanceControllerState`) rather than unconstrained LLM loops.
Fintech Payment Rails	**Razorpay Test-Mode SDK**	Native Indian Fintech standard for payment links, webhooks, and currency management.	Stripe / Cashfree	Razorpay is the benchmark API for Indian enterprise commerce and INR currency rails.
Frontend Architecture	**Tailwind CSS + Vanilla JS SPA**	Zero build-step overhead, sub-millisecond DOM rendering, dark-mode fintech UI.	React / Next.js	Eliminates Webpack/Vite compilation complexity, allowing direct static serving from FastAPI with instant load times.
Data Visualization	**Chart.js (v4.4)**	Canvas-based high-performance rendering for dynamic 90-day time-series trajectories.	Recharts / D3.js	Lightweight, responsive, and easily updated via JavaScript array manipulation without heavy virtual DOM overhead.
Data Storage Layer	**In-Memory Synchronized Store + JSON Fixtures**	Instant sub-millisecond reads/writes for live Digital Twin simulations.	PostgreSQL / SQLite	Perfect for hackathon demonstration and state cloning; designed with service interfaces for drop-in PostgreSQL migration.

PART 6 — SYSTEM ARCHITECTURE & END-TO-END PIPELINE



PART 11 — ROLE-BASED ACCESS CONTROL (RBAC) MATRIX

FinPilot AI implements strict 4-role enterprise security enforced at the **FastAPI dependency layer**, not merely the frontend UI.

Resource / Operation	👑 CFO (`CFO-001`)	💼 Finance Mgr (`FIN-MGR-001`)	🛠️ Dept Head (`ENG-HEAD-001`)	🔍 Auditor (`AUDITOR-001`)
Company Cash & Health Score	Full Visibility	Full Visibility	Aggregated Score Only	Full Visibility (Audit)
Department Budgets	All 5 Departments	All 5 Departments	**Own Department Only**	All 5 Departments
Invoice Management	View & Final Sign-Off	View & Verify Invoices	**Submit & View Own**	Read-Only Audit & Flag
Disbursement Approval ($\ge ₹50K$)	**Authorize / Reject**	Authorize up to ₹50K	Denied	**Denied (Flagging Only)**
Digital Twin What-If Studio	Full Forward Simulation	Full Forward Simulation	Scoped Expense Sim	Read-Only Trajectory
Payroll & Form 16	Full Access	Full Access	Own Team Basic Stats	Full Access & Anomaly Flag
Targeted Notifications	User-Scoped + CC	User-Scoped + CC	User-Scoped Only	User-Scoped Only
Immutable Audit Trail	View All Logs	View All Logs	Denied	**View All + Integrity Check**

> ⚠️ **Why Frontend-Only RBAC is Insecure:**
> If role checks only hide buttons in HTML/CSS, an attacker can easily bypass the UI by calling the backend API directly (e.g. via `curl` or Postman). In FinPilot AI, `get_current_user` in `src/app/core/auth_middleware.py` inspects the user token on every request, raising an `HTTP 403 Forbidden` if unauthorized.

PART 13 & 14 — LANGGRAPH & MULTI-AGENT ARCHITECTURE

Why LangGraph Instead of Simple LLM Calls?

Simple LLM API calls are stateless and non-deterministic. Financial decision-making requires **stateful cyclic graphs** where execution can pause for human approval, recover from tool errors, and pass structured memory between sub-agents.

The 7 Specialized Sub-Agents in FinPilot AI:

1. **Intent & RBAC Gatekeeper:** Classifies intent (What-If, Causal, Affordability) and enforces role scope.
2. **Retrieval Agent:** Queries in-memory ledger state and maintains strict data lineage.
3. **Analysis Agent:** Deterministically calculates burn velocity, GST amounts, and variance percentages.
4. **Risk Agent:** Evaluates weighted risk across 4 key financial metrics.
5. **Simulation Agent:** Branches the Financial Digital Twin and models 90-day cash trajectories.
6. **Causal Agent:** Correlates expenditure spikes with operational events (e.g. cluster launches).
7. **Narrator Agent:** Generates structured 5-part executive synthesis reports.

PART 16 — DETERMINISTIC FINANCIAL LOGIC VS. PROBABILISTIC LLM

> 🎓 **Critical Interview Topic: When to Use AI vs. When NOT to Use AI**
> This is one of the most important concepts interviewers look for. You must demonstrate that you know the limits of LLMs.

🚫 DETERMINISTIC PYTHON CODE (No LLM)

- **Statutory 18% GST:** `round(subtotal * 0.18, 2)`
- **Budget Balance Math:** `alloc - spent - pending`
- **PF & TDS Withholdings:** 12% basic salary exact math.
- **HITL ₹50,000 Threshold:** Validated in Pydantic.
- **Spending Token Limits:** Enforced in middleware.
- **Cryptographic Hashing:** SHA-256 block generation.

🤖 PROBABILISTIC AI AGENTS (LLMs)

- **Natural Language Intent Parsing:** Understanding freeform cart requests (*"Add 2 gateways & pay"*).
- **Causal Correlation:** Correlating unstructured commit logs and vendor bursts with budget spikes.
- **Counterfactual Narration:** Synthesizing 90-day simulation curves into 5-part executive summaries.
- **Autonomous Negotiation:** AI-to-AI price bargaining within pre-approved discount ranges.

PART 25 — RAZORPAY TEST-MODE FINTECH INTEGRATION

FinPilot AI integrates directly with **Razorpay test-mode APIs** (`src/app/services/razorpay_service.py`) for live payment link dispatch:

1. Payment Link Generation (`create_payment_link`):

Converts calculated INR amounts to smallest currency unit (paise: `int(amount * 100)`), generates short URLs (`https://rzp.io/i/...`), and attaches reference tags.

2. Webhook Signature Verification (`verify_webhook_signature`):

Computes HMAC-SHA256 digests over incoming webhook payloads using the secret key, preventing spoofed payment confirmations.

3. Idempotency & Replay Protection:

Attaches unique `X-Idempotency-Key` headers on all payment requests, ensuring that network retries do not trigger duplicate charges.

PART 54 — TOP INTERVIEW QUESTIONS & MODEL ANSWERS

Q1: How does FinPilot AI prevent LLM hallucinations in financial calculations?

⭐⭐⭐⭐⭐

Answer: We enforce an architectural barrier between reasoning and arithmetic. All mathematical calculations — such as 18% GST tax, budget utilization percentages, salary deductions, and runway continuity — are executed in deterministic Python functions. The LLM is never prompted to compute mathematical sums; it only receives pre-calculated numbers to formulate semantic explanations.

Q2: How does the AI-to-AI Shopping protocol work? ⭐⭐⭐⭐⭐

Answer: Our merchant service serves a machine-readable JSON-LD manifest at `/v1/commerce/ai-manifest`. External buyer bots inspect this catalog and send autonomous purchase requests to `/v1/commerce/ai-buy` along with an authorized `X-Spending-Token-Limit` header. If the total is within bounds, our merchant agent negotiates tiered volume discounts (up to 15%) and generates a Razorpay payment link.

Q3: What is the Financial Digital Twin and how does it simulate 90 days forward?

⭐⭐⭐⭐⭐

Answer: The Digital Twin is an in-memory synchronized clone of enterprise cash reserves, department allocations, daily burn rates, and vendor liabilities (`digital_twin_service.py`). It applies day-by-day differential equations ($Cash_i = Cash_{i-1} + Inflows_i - Outflows_i$) across 90 steps with custom multipliers (like 15% burn spikes or 14-day vendor payment deferrals), predicting cash runways without altering production ledgers.

Q4: Why use LangGraph instead of calling the LLM directly? ⭐⭐⭐⭐

Answer: Direct LLM calls are stateless and lack deterministic control. LangGraph provides a cyclic state machine (`FinanceControllerState`) with checkpointing and native Human-in-the-Loop interrupts. When a disbursement exceeds ₹50,000, LangGraph pauses execution at an approval node, awaits manager authorization, and only resumes after sign-off.

Q5: How is role-based notification isolation enforced? ⭐⭐⭐⭐

Answer: Rather than relying on frontend filters, `notification_service.py` executes query-level filtering against `recipientUserId == current_user.id` or explicit `observerIds`. If a CFO sends a request to Finance Manager A, Finance Manager B's API query returns zero results for that notification.

Q6: How do you handle Razorpay API timeouts or network failures? ⭐⭐⭐⭐

Answer: Our 4-Tier Failure Resilience Engine uses exponential backoff retry with idempotency keys. If all 3 retries fail, the transaction is safely transitioned to an `OFFLINE_QUEUED` state, a high-priority notification is dispatched to the finance manager, and ledger balances remain untouched.

PART 64 — ONE-PAGE EMERGENCY REVISION CHEAT SHEET

📌 **Core Architecture:**
- **Backend:** FastAPI, Python 3.11, Pydantic v2.
- **AI Engine:** LangGraph Multi-Agent, MemorySaver.
- **Fintech Rails:** Razorpay Test-Mode SDK.
- **Frontend:** Tailwind CSS, Chart.js, Vanilla JS SPA.

💉 **Digital Twin & Metrics:**
- **Enterprise Liquidity:** ₹8,435,000.00
- **Decision Health Score:** 82/100 (Healthy)
- **Departments:** Eng, Mkt, Sales, Ops, HR
- **Simulation Window:** 90 Days Step Differential

🤖 **7 Reasoning Sub-Agents:**
1. Intent & RBAC Agent 2. Retrieval Agent
3. Analysis Agent 4. Risk Agent
5. Simulation Agent 6. Causal Agent
7. Narrator Agent (5-Part Report Synthesis)

🛡️ **Safety & Governance:**
- **HITL Threshold:** $\ge ₹50,000$ requires CFO sign-off.
- **Audit Ledger:** Cryptographic SHA-256 chaining.
- **Security:** Recipient Isolation (`recipientUserId`).
- **A2A Ceilings:** `X-Spending-Token-Limit` bounds.

Part 75 — Final Project Synthesis

FinPilot AI is a production-grade demonstration of how modern autonomous AI agents can be combined with deterministic financial guardrails to transform corporate finance from retrospective reporting to proactive simulation.

FinPilot AI • Built for Razorpay AI Agent Track • Open Source on **GitHub**